

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: CSA0714 COURSE CODE: Computer Networks

COURSE OUTCOME COVERED

CO1: *Apply the functions of OSI layer in enabling reliable data transmission across networks. (BL2, BL3)*

Assessment Tool Weightage: 40%

Annexure A

SHORT ANSWER TEST

Code of Conduct:

*I Chiranjibi Pradhan 192511475(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Annexure A

Short Answer Test

1. A company needs to transfer a **150 MB** design file from a client computer to a remote server over a network with a bandwidth of **50 Mbps**. The Transport Layer divides the file into **1500-byte segments**, while the Data Link Layer encapsulates each segment into frames by adding **40 bytes of header and trailer**. Calculate the total number of segments and frames transmitted, the total transmission overhead introduced by the Data Link Layer, and estimate the total transmission time (ignoring propagation delay). Explain how the Transport and Data Link layers ensure reliable communication.
2. A Transport Layer protocol uses a **sliding window size of 6 frames**. Each frame carries **1024 bytes** of user data. If the sender transmits **48 frames**, calculate the number of transmission windows required. If one frame in every window is lost and must be retransmitted, determine the total number of retransmissions. Explain how flow control improves communication efficiency.
3. A **12,000-byte** IP packet is transmitted through a router where the Maximum Transmission Unit (MTU) is **1500 bytes**. Assuming each fragment requires a **20-byte IP header**, calculate the total number of fragments created and the total header overhead introduced. Explain how the Network Layer ensures successful delivery of fragmented packets to the destination.
4. A Transport Layer protocol uses a **sliding window size of 6 frames**. Each frame carries **1024 bytes** of user data. If the sender transmits **48 frames**, calculate the number of transmission windows required. If one frame in every window is lost and must be retransmitted, determine the total number of retransmissions. Explain how flow control improves communication efficiency.

5. A multimedia application transfers a **600 MB** video file. The Session Layer inserts a synchronization checkpoint after every **60 MB** of transmitted data. During transmission, a failure occurs after **390 MB** has been transferred. Determine the number of checkpoints created before failure and calculate the amount of data that must be retransmitted after recovery. Explain the importance of synchronization in the Session Layer.

6. Before transmission, the Presentation Layer compresses a **900 MB** file by **40%**. The compressed data is encrypted, adding **5%** overhead to the compressed size. Calculate the compressed file size, the encrypted file size, and the total percentage reduction compared to the original file. Explain how the Presentation Layer improves transmission efficiency and security.

7. An FTP application transfers **3 GB** of data over a network with an effective throughput of **120 Mbps**. The Application Layer divides the data into requests of **3 MB** each. Calculate the total number of requests generated and estimate the total transmission time in seconds. Discuss how the Application Layer provides reliable services to users.

8. A packet experiences the following processing delays at different OSI layers: Application Layer = **4 ms**, Transport Layer = **3 ms**, Network Layer = **5 ms**, Data Link Layer = **2 ms**, and Physical Layer = **1 ms**. The propagation delay is **18 ms**, and the transmission delay is **12 ms**. Calculate the total end-to-end delay experienced by the packet and explain how each layer contributes to the communication process.

9. A network transfers **80 MB** of useful data using frames of **1600 bytes**, where each frame contains **80 bytes** of protocol overhead added by different OSI layers.

Calculate the total number of frames required, the total overhead transmitted, and the transmission efficiency as a percentage. Explain how protocol overhead affects network performance.

10. A hospital transfers a patient's medical record of **240 MB** through an IoT-enabled healthcare network. The Presentation Layer compresses the data by **30%**, the Transport Layer divides the compressed data into **1400-byte segments**, and the Data Link Layer adds **60 bytes** of overhead to each frame. Calculate the compressed file size, the total number of segments transmitted, the total protocol overhead introduced, and the final amount of data transmitted across the Physical Layer. Explain how multiple OSI layers work together to ensure secure and reliable communication.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates a thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers

		and coherence			
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandabl e with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

Computer Networks – Assessment Tool 1

Question 1

Given

File size = 150 MB, bandwidth = 50 Mbps, transport-layer segment = 1500 bytes, data link overhead = 40 bytes per frame (header + trailer). Propagation delay is ignored.

Working

Taking 150 MB as 150,000,000 bytes, the number of segments is $150,000,000 / 1500 = 100,000$ segments. Since each segment becomes exactly one frame at the data link layer, the number of frames is also 100,000.

Overhead added by the data link layer = $100,000 \text{ frames} \times 40 \text{ bytes} = 4,000,000 \text{ bytes}$, i.e. 4 MB. So the actual amount of data placed on the wire = $150,000,000 + 4,000,000 = 154,000,000 \text{ bytes}$.

Converting to bits: $154,000,000 \times 8 = 1,232,000,000 \text{ bits}$. Transmission time = $1,232,000,000 \text{ bits} / 50,000,000 \text{ bps} \approx 24.64 \text{ seconds}$.

Explanation

Reliability here comes from two layers working together. The transport layer breaks the file into numbered segments and keeps track of which ones have reached the destination — if an acknowledgement doesn't come back in time, it resends that segment. This is exactly what TCP does, along with basic flow control so the sender doesn't flood a slower receiver.

The data link layer's job is more local: it wraps each segment in a frame and adds a trailer (often a CRC checksum) so the receiving node on that particular link can tell if the frame arrived intact. If a frame is damaged in transit, it can be discarded and, depending on the protocol in use, retransmitted between just those two devices rather than end-to-end. Together, these two layers catch errors at different scopes — link-level and end-to-end — which is what makes the overall transfer dependable even though the underlying physical medium is not perfectly reliable.

Question 2

Given

Sliding window size = 6 frames, each frame carries 1024 bytes of data, sender transmits 48 frames in total, and exactly one frame per window is lost.

Working

Number of windows needed = $\text{total frames} / \text{window size} = 48 / 6 = 8 \text{ windows}$.

Since one frame is lost in every window and each lost frame needs one retransmission, total retransmissions = $8 \text{ windows} \times 1 \text{ lost frame} = 8 \text{ retransmissions}$.

Explanation

A sliding window lets the sender push out several frames before it has to stop and wait for an acknowledgement, instead of sending one frame, waiting, sending the next, and so on. That single change is what keeps the link busy and pushes throughput up, especially when the round-trip time is large compared to the time it takes to transmit a frame.

At the same time, the window size puts a ceiling on how many unacknowledged frames can be outstanding, so a fast sender can't overrun a receiver that has limited buffer space — this is the flow-control part. When a frame within the window is lost, only that portion of the sequence needs to be resent (in protocols such as Selective Repeat), so the loss doesn't force the whole transfer to restart from scratch.

Question 3

Given

IP packet size = 12,000 bytes, path MTU = 1500 bytes, and each fragment carries its own 20-byte IP header.

Working

Usable payload per fragment = MTU – header = 1500 – 20 = 1480 bytes.

Number of fragments = 12,000 / 1480 $\approx$ 8.11, which rounds up to 9 fragments (the last fragment carries the leftover 160 bytes).

Total header overhead = 9 fragments $\times$ 20 bytes = 180 bytes.

Explanation

Fragmentation happens because different links along a path can have different MTUs, and the network layer has to shrink the packet to fit whichever link is the bottleneck. Each fragment gets its own IP header, and that header carries an identification field (so all pieces of the same original packet can be tied back together), a fragment offset (so the receiver knows where each piece belongs in the original data stream), and a more-fragments flag.

The destination host buffers incoming fragments that share the same identification field until it has all of them, uses the offsets to put them back in order, and then reassembles the original 12,000-byte packet before handing it up to the transport layer. If any fragment is missing, the whole packet is normally dropped and has to be resent by the source, which is one reason excessive fragmentation is generally avoided where possible.

Question 4

Given

Sliding window size = 6 frames, each frame carries 1024 bytes of data, sender transmits 48 frames in total, and exactly one frame per window is lost. (Same parameters as Question 2.)

Working

Number of windows needed = 48 / 6 = 8 windows.

Total retransmissions = 8 windows $\times$ 1 lost frame per window = 8 retransmissions.

Explanation

As explained above, sending frames in windows rather than one at a time keeps the link utilised while a round trip is in progress, which is the main efficiency gain that flow control provides.

The window also protects the receiver — it can only be sent as many frames as its buffer is known to handle, so it never gets overwhelmed. When a loss occurs inside a window, the retransmission is limited to that lost frame (or, depending on the scheme, everything from that point in the window onward), so the cost of an error stays proportional to the window size rather than to the whole file.

Question 5

Given

Video file = 600 MB, a synchronisation checkpoint is inserted every 60 MB, and the transfer fails after 390 MB have gone through.

Working

Checkpoints are placed at 60, 120, 180, 240, 300 and 360 MB. Since failure occurs at 390 MB, only the checkpoint at 360 MB has been passed — that's 6 checkpoints reached before the failure.

Data lost since the last checkpoint = $390 - 360 = 30$ MB, so that is the amount that has to be retransmitted once the connection recovers.

Explanation

The session layer is responsible for keeping a long-running exchange organised, and checkpointing is one of its main tools for doing that. By marking off the data stream every 60 MB, it gives both ends a shared reference point they can fall back to if something goes wrong partway through.

Without this, a failure at the 390 MB mark would mean starting the whole 600 MB transfer over again. With checkpointing, recovery only has to redo the 30 MB sent after the last checkpoint, which is a much smaller cost in time and bandwidth — this is particularly valuable for large multimedia transfers where restarting from zero every time a connection drops would be impractical.

Question 6

Given

Original file = 900 MB. The presentation layer compresses it by 40%, and encryption then adds a further 5% overhead on top of the compressed size.

Working

Compressed size = $900 \times (1 - 0.40) = 900 \times 0.60 = 540$ MB.

Encrypted size = $540 \times (1 + 0.05) = 540 \times 1.05 = 567$ MB.

Overall reduction compared to the original = $(900 - 567) / 900 \times 100 = 37\%$.

Explanation

Compression and encryption are both handled at the presentation layer, but they pull in opposite directions in terms of size — compression removes redundancy to shrink the file, while encryption typically pads or restructures data in a way that adds a small amount back. Even so, the net effect here is still a 37% reduction from the original 900 MB, so the transfer is noticeably lighter on the network than sending the file raw.

Beyond the size benefit, this layer is also where data gets converted into a common representation both ends agree on (encoding, formatting, and so on), so compression and encryption sit alongside that translation role. The security gained from encrypting the already-compressed data means that even if the transmission were intercepted, the content would not be readable without the key, which matters a lot for anything sensitive.

Question 7

Given

Total data to transfer = 3 GB, effective throughput = 120 Mbps, and the application layer splits the transfer into 3 MB requests.

Working

Taking 3 GB as 3000 MB, the number of requests = $3000 / 3 = 1000$ requests.

Total data in bits = $3,000,000,000 \text{ bytes} \times 8 = 24,000,000,000$ bits. Transmission time = $24,000,000,000 / 120,000,000 = 200$ seconds.

Explanation

Breaking a large transfer into fixed-size requests, as FTP does, gives the application layer finer control over the exchange — each request can be tracked, acknowledged, and retried individually if something goes wrong, rather than treating the entire 3 GB as one indivisible unit.

This is also where user-facing reliability really comes from: things like login/authentication, directory and file-handling commands, and status codes for each request are all application-layer concerns. The lower layers still do the actual work of getting bits across the network, but the application layer is what turns that raw connectivity into a usable, dependable file-transfer service for the end user.

Question 8

Given

Processing delays — Application 4 ms, Transport 3 ms, Network 5 ms, Data Link 2 ms, Physical 1 ms. Propagation delay = 18 ms, transmission delay = 12 ms.

Working

Sum of processing delays across the five layers = $4 + 3 + 5 + 2 + 1 = 15$ ms.

Total end-to-end delay = 15 ms (processing) + 18 ms (propagation) + 12 ms (transmission) = 45 ms.

Explanation

Every layer a packet passes through adds a small amount of processing time as it wraps or unwraps headers, checks addresses, or verifies checksums — that's the 15 ms total here. On top of that, two more delays come from the physical act of sending the packet: transmission delay is how long it takes to push all the bits of the packet onto the link, and propagation delay is how long the signal takes to physically travel across the medium once it's on the wire.

None of these delays are optional — they happen even in a perfectly functioning network — which is why real-world latency is always the sum of processing, transmission and propagation, not any single one of them. In this case the 45 ms figure gives a realistic sense of how each layer, plus the physical characteristics of the link, contributes to the overall delay a user would notice.

Question 9

Given

Useful data to send = 80 MB, each frame is 1600 bytes in size, and 80 bytes of that frame is protocol overhead added by the various OSI layers.

Working

Usable data per frame = $1600 - 80 = 1520$ bytes.

Number of frames = $80,000,000 / 1520 \approx 52,631.6$, rounded up to 52,632 frames.

Total overhead transmitted = $52,632 \times 80 = 4,210,560$ bytes. Total bytes actually sent = $52,632 \times 1600 = 84,211,200$ bytes.

Transmission efficiency = $\text{useful data} / \text{total data} \times 100 = 80,000,000 / 84,211,200 \times 100 \approx 95\%$.

Explanation

Overhead is unavoidable — headers and trailers are what carry addressing, sequencing and error-checking information — but it does eat into the frame's capacity for actual user data. Here, 80 out of every 1600 bytes (5%) is overhead, which is why the efficiency works out to roughly 95% rather than 100%.

The general pattern is that smaller frames carry proportionally more overhead relative to payload, which hurts efficiency, while very large frames reduce that overhead ratio but increase the cost of retransmitting a whole frame if

even a single bit gets corrupted. Network designers choose frame sizes with this trade-off in mind, aiming for a size that keeps overhead low without making errors expensive to recover from.

Question 10

Given

Medical record = 240 MB. The presentation layer compresses it by 30%, the transport layer splits the compressed data into 1400-byte segments, and the data link layer adds 60 bytes of overhead per frame.

Working

Compressed size = $240 \times (1 - 0.30) = 240 \times 0.70 = 168$ MB.

Taking 168 MB as 168,000,000 bytes, number of segments = $168,000,000 / 1400 = 120,000$ segments.

Data link overhead = $120,000 \times 60 = 7,200,000$ bytes, i.e. 7.2 MB.

Data finally sent over the physical layer = $168,000,000 + 7,200,000 = 175,200,000$ bytes, i.e. 175.2 MB.

Explanation

This scenario is a good illustration of how the layers stack their work on top of each other rather than acting in isolation. The presentation layer shrinks the sensitive record first — and in a real IoT healthcare setting would very likely encrypt it too — before it's handed down to the transport layer, which chops the now-smaller file into manageable segments and takes responsibility for making sure they all arrive.

The data link layer then frames each of those segments for the specific hop it's crossing, adding its own overhead for addressing and error checking, and the physical layer is what finally puts the resulting 175.2 MB of framed data onto the network as raw bits. Each layer only worries about its own part of the job, but the combination is what gets a patient's record across an IoT healthcare network both securely and reliably.
